Homework 4: Shor's Algorithm and the IBM $N = 21$ Demonstration

Quantum Computing Training 2026

Overview

This homework set accompanies the lecture on Shor's algorithm. The goals are to understand:

(i) how factoring reduces to order finding;

(ii) how QPE extracts information about the order;

(iii) how the classical continued-fractions and greatest-common-divisor steps finish the factorization;

(iv) what is demonstrated, simplified, and compiled in a real small-scale experiment.

The required paper reading is:

> U. Skosana and M. Tame, "Demonstration of Shor's factoring algorithm for $N = 21$ on IBM quantum processors," *Scientific Reports* **11**, 16599 (2021). DOI: 10.1038/s41598-021-95973-w.

The lecture notebook uses the textbook-style order-finding example

$$N = 21, \qquad g = 2,$$

for which the order is $r = 6$. The paper uses a compiled experimental instance

$$N = 21, \qquad a = 4,$$

for which the order is $r = 3$. A major theme of this assignment is to compare these two examples.

Problem 1: From Factoring to Order Finding

Part A: The lecture instance $N = 21$, $g = 2$

Let

$$N = 21, \qquad g = 2.$$

A1. Compute $\gcd(g, N)$. Does this immediately reveal a nontrivial factor of N?

A2. Compute

$$2^1, 2^2, 2^3, \ldots \pmod{21}$$

until the sequence first returns to 1. What is the order

$$r = \mathrm{ord}_{21}(2)?$$

A3. Verify that r is even and that

$$2^{r/2} \not\equiv -1 \pmod{21}.$$

A4. Compute

$$\gcd(2^{r/2} - 1, 21), \qquad \gcd(2^{r/2} + 1, 21),$$

and recover the factorization of 21.

Part B: The paper instance $N = 21$, $a = 4$

The IBM paper uses a compiled circuit for

$$N = 21, \qquad a = 4.$$

B1. Compute $\gcd(a, N)$.

B2. Compute the order

$$r = \mathrm{ord}_{21}(4).$$

B3. Explain why the usual textbook post-processing condition "r is even" fails for this instance.

B4. Nevertheless, note that $a = 4 = 2^2$ is a perfect square. Let $b = 2$. Show that

$$a^r = b^{2r} \equiv 1 \pmod{21}.$$

Then compute

$$\gcd(b^r - 1, 21), \qquad \gcd(b^r + 1, 21).$$

B5. In one paragraph, explain why the paper's $a = 4$ instance is mathematically related to Shor's algorithm but is not the same as choosing a generic random base g and hoping for the usual even-order condition.

Problem 2: Simulating Textbook Order Finding for $N = 21$

In this problem you will simulate the lecture-style QPE order-finding subroutine for

$$N = 21, \qquad g = 2.$$

Let

$$n = \lceil \log_2 N \rceil = 5$$

be the number of work qubits, and let m be the number of counting qubits. Use

$$m \in \{6, 8, 10\}.$$

Define the modular multiplication unitary

$$U_g \, |y\rangle = |gy \bmod N\rangle$$

for $0 \leq y < N$. For computational basis states $|y\rangle$ with $N \leq y < 2^n$, you may define $U_g \, |y\rangle = |y\rangle$ so that U_g is a valid permutation on all 2^n basis states.

1. Write a function `modular_multiply_gate(g, N, n_work)` that returns a Qiskit gate implementing $|y\rangle \mapsto |gy \bmod N\rangle$ for $y < N$ and $|y\rangle \mapsto |y\rangle$ for $y \geq N$.
2. Build the QPE/order-finding circuit with:
   - m counting qubits initialized to $|+\rangle^{\otimes m}$;
   - $n = 5$ work qubits initialized to $|1\rangle$;
   - controlled powers $U_g^{2^j}$;
   - an inverse QFT on the counting register;
   - measurement of the counting register.
3. For each $m \in \{6, 8, 10\}$, simulate the circuit and plot the probability distribution over measured integers $y \in \{0, \ldots, 2^m - 1\}$.
4. For the ten largest-probability outcomes y, compute
   $$\frac{y}{2^m}$$
   and use continued fractions to find candidate denominators r' satisfying $r' \leq N$.
5. For each candidate r', test whether
   $$2^{r'} \equiv 1 \pmod{21}.$$
6. If a valid order is found, use the gcd step to factor 21.
7. The ideal eigenphases are k/r for $k = 0, 1, \ldots, r - 1$. For $r = 6$, list these phases.
8. For each value of m, what measured integers y are closest to
   $$2^m \frac{k}{6}?$$
9. Which values of k are useful for recovering $r = 6$ by continued fractions? Which are not useful? Explain using $\gcd(k, r)$.
10. How does increasing m change the sharpness of the peaks and the reliability of continued fractions?

Problem 3: Guided Reading of the IBM $N = 21$ Demonstration

Read the Skosana–Tame paper carefully.

Part A: Resource comparison

A1. For $N = 21$, compute the lecture-style register sizes:

$$n = \lceil \log_2 21 \rceil, \qquad m = 2n,$$

and the total number $m+n$ of main-register qubits before any additional arithmetic workspace.

A2. According to the paper, how many total qubits are used in the compiled IBM implementation? How many are control qubits and how many are work qubits?

A3. The paper states that a full-scale implementation for $N = 21$ would require far more gates/qubits than the experiment used. Summarize this comparison in your own words.

Part B: What was compiled?

B1. In textbook Shor, modular exponentiation implements

$$|x\rangle\,|1\rangle \longmapsto |x\rangle\,|a^x \bmod N\rangle\,.$$

Does the IBM experiment implement this operation generically for arbitrary N and a?

B2. Identify at least two simplifications that use knowledge of the special instance $N = 21$, $a = 4$.

B3. For each simplification, say whether it would still be available if the goal were to factor a large unknown RSA modulus.

Part C: Interpreting the measured distribution

For the paper's compiled instance, use

$$N = 21, \qquad a = 4, \qquad r = 3,$$

and three control qubits, so

$$Q = 2^3 = 8.$$

C1. The ideal phases are

$$\frac{k}{r}, \qquad k = 0, 1, 2.$$

List these values.

C2. Compute the ideal peak locations

$$y \approx Q\frac{k}{r}$$

for $k = 0, 1, 2$.

C3. Explain why the bit strings 000, 011, and 101 are natural high-probability outcomes for a three-bit control register.

C4. Apply continued fractions to

$$\frac{3}{8}, \qquad \frac{5}{8}, \qquad \frac{6}{8}.$$

Which outcomes recover $r = 3$? Which outcome gives a misleading candidate?

C5. Explain why adding more control qubits would make the peaks narrower and make the continued-fractions step more reliable.

Part D: Balanced interpretation

Write one paragraph summarizing: What part of Shor's algorithm was genuinely demonstrated? What was compiled or simplified? Why is the experiment scientifically meaningful? Why does it not imply that today's quantum computers can factor cryptographic RSA numbers?

Bonus: Toffoli Gates, Relative Phases, and When Phases Matter

The IBM paper reduces circuit cost by using approximate or relative-phase Toffoli gates. These gates can preserve the classical truth-table action while changing phases on some computational basis states. The standard Toffoli gate acts as

$$|a, b, c\rangle \longmapsto |a, b, c \oplus ab\rangle \, .$$

Reversible arithmetic often creates temporary "scratch work." Work through the following toy example to see why this workspace has to be cleaned up. Let

$$f(x_1, x_2) = x_1 x_2.$$

This is the Boolean AND function of the two input bits.

Part A: Reversible classical logic

A1. Explain why ordinary irreversible classical arithmetic cannot simply be inserted into a quantum circuit. (Hint: Think about reversibility/unitarity)

A2. A Toffoli gate maps

$$|a, b, c\rangle \mapsto |a, b, c \oplus ab\rangle.$$

If $c = 0$, show that the third bit becomes $f(a, b)$.

A3. Now suppose the two input qubits are in superposition:

$$|\psi\rangle = \frac{|00\rangle + |01\rangle + |10\rangle + |11\rangle}{2}.$$

To apply a Toffoli gate, add a third qubit initialized to $|0\rangle$. The full input state is

$$|\psi\rangle|0\rangle = \frac{|00\rangle|0\rangle + |01\rangle|0\rangle + |10\rangle|0\rangle + |11\rangle|0\rangle}{2}.$$

Apply a Toffoli gate with the first two qubits as controls and the third qubit as the target. What is the resulting three-qubit state?

A4. In your answer to part (b), the third qubit is $|0\rangle$ for three branches and $|1\rangle$ for one branch. Which input branch is correlated with the ancilla state $|1\rangle$?

A5. Suppose the goal was only to use the ancilla temporarily, for example to apply the phase

$$|x_1x_2\rangle \mapsto (-1)^{f(x_1,x_2)}|x_1x_2\rangle.$$

Why would leaving the ancilla entangled with the input register be dangerous for later interference?

A6. What operation would you apply to return the ancilla to $|0\rangle$?

A7. Explain how this toy example relates to modular exponentiation in Shor's algorithm. In particular, why do reversible adders and multipliers need carry bits, comparison bits, or partial-product bits, and why must these be uncomputed?

Part B: A toy relative-phase Toffoli model

Define a toy three-qubit gate $RCCX$ by

$$RCCX\,|a,b,c\rangle = (-1)^{abc}\,|a,b,c\oplus ab\rangle\,.$$

This gate has the same classical input-output map as Toffoli, but it introduces a phase of -1 on the basis state $|111\rangle$.

B1. Write the 8×8 matrix for the exact Toffoli gate and for this toy $RCCX$ gate, using computational basis order

$$|000\rangle\,, |001\rangle\,, \ldots, |111\rangle\,.$$

B2. Verify that the two gates give the same measurement results when they act on each computational basis input individually.

B3. Find an input superposition for which the two gates produce different output states.

B4. Explain why preserving a classical truth table does not fully specify a quantum unitary gate.

Part C: Why residual phases may or may not be harmless

C1. In Shor's algorithm, quantum phase estimation converts relative phases into measurement probabilities. Why might an unintended phase inside the modular multiplication circuit change the observed order-finding peaks?

C2. The IBM $N = 21$ experiment uses a heavily compiled circuit for one small instance. The work register only explores a small, known set of states. Why does this make it possible to check directly that the relative phases are harmless for this particular experiment?

C3. Why can relative-phase Toffoli gates be useful in the compiled IBM $N = 21$ experiment, but dangerous if used carelessly in a general Shor implementation?